Product Microservice (`product-service`)

Complete Technical Specification, Database Architecture & API Documentation

PLATFORM	DATABASE ENGINE	SERVICE PORT	VERSION
Distributed E-Commerce	PostgreSQL (`product_db`)	8083	1.0.0 (August 2026)

1. Executive Summary & Objective

The **Product Microservice (`product-service`)** is an essential core service of the Distributed E-Commerce platform. It manages the central product catalog, item inventory details, category classifications, and keyword searches. It facilitates seamless integration with other microservices (such as `cart-service` and `inventory-service`) using OpenFeign and Eureka Service Registry.

- **Central Catalog Management**: Full CRUD operations for product items.
- **PostgreSQL Relational Persistence**: High-reliability storage using Spring Data JPA with PostgreSQL.
- **RESTful API Interface**: Versioned APIs exposed under `/api/v1/products`.
- **Spring Security & JWT**: Role-based authorization protecting administrative mutative endpoints.

2. Technology Stack & Dependencies

TECHNOLOGY / LIBRARY	VERSION	DESCRIPTION / ROLE
Java	21 (LTS)	Programming language platform
Spring Boot	3.2.5	Application microservice framework
Spring Data JPA	3.2.5	ORM and database persistence layer
PostgreSQL Driver	Runtime	JDBC relational database connectivity
Spring Security	3.2.5	Security filter chain and authentication
JJWT (io.jsonwebtoken)	0.12.7	JWT token verification & parsing
Spring Cloud Eureka Client	2023.0.1	Microservice registration and discovery
Spring Cloud OpenFeign	2023.0.1	Declarative HTTP client for inter-service calls
MapStruct	1.5.5.Final	Type-safe bean mapping between Entity and DTOs
Lombok	1.18.30	Boilerplate reduction (@Data, @Builder)
SpringDoc OpenAPI / Swagger UI	2.3.0	Automated interactive API documentation

3. Database Architecture & Schema Specification

The service utilizes a dedicated PostgreSQL database named **`product_db`** and stores product records in the **`products`** table.

Table Name: **`products`**

COLUMN NAME	DATA TYPE	CONSTRAINTS	NULLABLE	DESCRIPTION
`id`	`BIGINT`	`PRIMARY KEY`, Auto Increment	No	Unique Product Identifier
`name`	`VARCHAR(255)`	`NOT NULL`	No	Product title / name

COLUMN NAME	DATA TYPE	CONSTRAINTS	NULLABLE	DESCRIPTION
brand	VARCHAR(255)	-	Yes	Manufacturer or brand name
category	VARCHAR(255)	-	Yes	Category classification (e.g. Electronics)
description	TEXT	-	Yes	Detailed product description
price	NUMERIC(12,2)	CHECK (price > 0.0)	No	Unit price in decimal precision
stock	INTEGER	CHECK (stock >= 0)	No	Available inventory quantity
seller	VARCHAR(255)	-	Yes	Merchant / seller identifier
image_url	VARCHAR(255)	-	Yes	URL link to product image
active	BOOLEAN	DEFAULT TRUE	Yes	Availability status flag

SQL DDL Script

```
CREATE TABLE IF NOT EXISTS products (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    brand VARCHAR(255),
    category VARCHAR(255),
    description TEXT,
    price NUMERIC(12, 2) NOT NULL CHECK (price > 0.0),
    stock INT NOT NULL CHECK (stock >= 0),
    seller VARCHAR(255),
    image_url VARCHAR(255),
    active BOOLEAN DEFAULT TRUE
);
```

4. Class & Package Architecture Inventory

The codebase comprises **19 Java source files** across 10 specialized packages:

PACKAGE	CLASS / INTERFACE NAME	TYPE	KEY RESPONSIBILITY
com.ecommerce.productservice	ProductServiceApplication	Class	Spring Boot Main Entry Point
.config	OpenApiConfig	Class	Swagger/OpenAPI UI configuration
.config	SecurityConfig	Class	Spring Security Filter Chain & Permissions
.controller	ProductController	Class	REST Controller exposing `/api/v1/products`
.dto	ProductRequest	Class	Incoming JSON Payload with Jakarta Validations
.dto	ProductResponse	Class	Outgoing Product JSON DTO
.entity	Product	Class	JPA Entity for `products` table
.exception	ProductNotFoundException	Class	Runtime exception thrown when ID not found
.exception	ErrorResponse	Record	Standardized JSON error response payload

PACKAGE	CLASS / INTERFACE NAME	TYPE	KEY RESPONSIBILITY
`.exception`	`GlobalExceptionHandler`	Class	ControllerAdvice mapping exceptions to HTTP status
`.mapper`	`ProductMapper`	Interface	MapStruct bean mapper (Entity <-> DTO)
`.repository`	`ProductRepository`	Interface	JpaRepository providing custom SQL query methods
`.security`	`JwtAuthenticationFilter`	Class	Intercepts Requests & validates Bearer tokens
`.security`	`JwtService`	Class	Parses & validates HMAC secret JWTs
`.security`	`UserPrincipal`	Class	UserDetails wrapper for SecurityContext
`.security`	`CustomAccessDeniedHandler`	Class	JSON handler for 403 Forbidden errors
`.security`	`CustomAuthenticationEntryPoint`	Class	JSON handler for 401 Unauthorized errors
`.service`	`ProductService`	Interface	Service contract for product business logic
`.service.impl`	`ProductServiceImpl`	Class	Service implementation with repository calls

5. Detailed Methods Inventory

Controller Methods (`ProductController`)

- `ResponseEntity<ProductResponse> addProduct(@Valid @RequestBody ProductRequest request)`
- `ResponseEntity<List<ProductResponse>> getAllProducts()`
- `ResponseEntity<ProductResponse> getProductById(@PathVariable("id") Long id)`
- `ResponseEntity<ProductResponse> updateProduct(@PathVariable("id") Long id, @Valid @RequestBody ProductRequest request)`
- `ResponseEntity<Void> deleteProduct(@PathVariable("id") Long id)`
- `ResponseEntity<List<ProductResponse>> searchProducts(@RequestParam("keyword") String keyword)`
- `ResponseEntity<List<ProductResponse>> getProductsByCategory(@PathVariable("category") String category)`

Service Methods (`ProductServiceImpl`)

- `ProductResponse addProduct(ProductRequest request)` : Maps request DTO to Product entity, defaults active status to true, saves to PostgreSQL, returns mapped ProductResponse.
- `List<ProductResponse> getAllProducts()` : Fetches all product entities via JPA repository and maps to list of ProductResponse DTOs.
- `ProductResponse getProductById(Long id)` : Finds entity by primary key or throws `ProductNotFoundException` if absent.
- `ProductResponse updateProduct(Long id, ProductRequest request)` : Retrieves existing entity, updates non-null target fields using MapStruct, and saves back to DB.
- `void deleteProduct(Long id)` : Checks if ID exists and deletes record by ID.
- `List<ProductResponse> searchProducts(String keyword)` : Invokes `findByNameContainingIgnoreCase` in repository.
- `List<ProductResponse> getProductsByCategory(String category)` : Invokes `findByCategoryIgnoreCase` in repository.

6. Complete REST API Reference

API Endpoint Matrix

HTTP METHOD	ENDPOINT PATH	AUTHENTICATION REQUIRED	DESCRIPTION
POST	`/api/v1/products`	Yes (ADMIN / SELLER)	Create a new product record
GET	`/api/v1/products`	No (Public)	Get list of all active products
GET	`/api/v1/products/{id}`	No (Public)	Get single product details by ID
PUT	`/api/v1/products/{id}`	Yes (ADMIN / SELLER)	Update existing product details
DELETE	`/api/v1/products/{id}`	Yes (ADMIN)	Delete product by ID
GET	`/api/v1/products/search`	No (Public)	Search products by keyword query
GET	`/api/v1/products/category/{category}`	No (Public)	Filter products by category name

`POST` **`/api/v1/products`**

Description: Creates a new product record in the PostgreSQL database.

Request Headers: `Authorization: Bearer <JWT_TOKEN>` | `Content-Type: application/json`

Request Body Payload:

```
{
  "name": "Wireless Noise Cancelling Headphones",
  "brand": "AudioPro",
  "category": "Electronics",
  "description": "High fidelity over-ear bluetooth headphones with active noise cancellation.",
  "price": 199.99,
  "stock": 50,
  "seller": "TechWorld Store",
  "imageUrl": "https://images.example.com/products/headphones.jpg",
  "active": true
}
```

Response (HTTP 201 Created):

```
{
  "id": 1,
  "name": "Wireless Noise Cancelling Headphones",
  "brand": "AudioPro",
  "category": "Electronics",
  "description": "High fidelity over-ear bluetooth headphones with active noise cancellation.",
  "price": 199.99,
  "stock": 50,
  "seller": "TechWorld Store",
  "imageUrl": "https://images.example.com/products/headphones.jpg",
  "active": true
}
```

Error Response (HTTP 400 Bad Request):

```
{
  "timestamp": "2026-08-17T00:15:00",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed for object='productRequest'",
  "path": "/api/v1/products",
  "validationErrors": {
    "name": "Product name is required",
    "price": "Price must be greater than 0"
  }
}
```

`GET` **`/api/v1/products/{id}`**

Description: Fetches product details by ID.

Path Parameter: `id` (Long, e.g. `1`)

Response (HTTP 200 OK):


```
{
  "id": 1,
  "name": "Wireless Noise Cancelling Headphones",
  "brand": "AudioPro",
  "category": "Electronics",
  "description": "High fidelity over-ear bluetooth headphones with active noise cancellation.",
  "price": 199.99,
  "stock": 50,
  "seller": "TechWorld Store",
  "imageUrl": "https://images.example.com/products/headphones.jpg",
  "active": true
}
```


Error Response (HTTP 404 Not Found):


```
{
  "timestamp": "2026-08-17T00:15:00",
  "status": 404,
  "error": "Not Found",
  "message": "Product not found with ID: 999",
  "path": "/api/v1/products/999"
}
```


`GET` **`/api/v1/products/search?keyword={keyword}`**

Description: Searches products by performing case-insensitive matching on the product name.

Query Parameter: `keyword` (String, e.g. `Headphones`)

Response (HTTP 200 OK):


```
[
  {
    "id": 1,
    "name": "Wireless Noise Cancelling Headphones",
    "brand": "AudioPro",
    "category": "Electronics",
    "price": 199.99,
    "stock": 50,
    "active": true
  }
]
```